

Experiment No. – 3 B

Student Name: Navjot Kaur

UID: 25MCA20078

Branch: MCA

Section/Group: 1/A

Semester: 3rd

Date of Performance: 13-09-2026

Subject Name: Machine Learning

Subject Code: 25CAP-702

Aim/Overview of the practical: To implement Logistic Regression, k-Nearest Neighbors (k-NN) and Decision tree classification algorithms on a suitable dataset and evaluate their performance using Confusion Matrix, Precision, Recall, and F1-Score.

Objective:

1. Implement Logistic Regression, k-NN, and Decision Tree on the dataset.
2. Train and evaluate each using Confusion Matrix, Precision, Recall, and F1-Score.
3. Compare the three models and find the best one.

Apparatus Used:

- Google Colab
- Python
- Chrome Browser

Code for experiment:

1. Import Required Libraries

```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.linear_model import LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report
)
```

2. Upload the Dataset

```
uploaded = files.upload()
```

Choose Files adult.csv
adult.csv(text/csv) - 4104734 bytes, last modified: 19/9/2019 - 100% done
 Saving adult.csv to adult (1).csv

Figure 1: Successful Upload of the Adult Dataset

3. Load the Dataset

```
df = pd.read_csv("adult.csv")
```

4. Display first 5 enteries

```
df.head()
```

	age	workclass	fnlwgt	education	education.num	marital.status	occupation	relationship	race	sex	capital.gain	capital.loss	hours.per.week	native.country	incom
0	90	?	77063	HS-grad	9	Widowed	?	Not-in-family	White	Female	0	4356	40	United-States	<=50
1	82	Private	132870	HS-grad	9	Widowed	Exec-managerial	Not-in-family	White	Female	0	4356	18	United-States	<=50
2	66	?	186061	Some-college	10	Widowed	?	Unmarried	Black	Female	0	4356	40	United-States	<=50
3	54	Private	140388	7th-8th	4	Divorced	Machine-op-inspct	Unmarried	White	Female	0	3900	40	United-States	<=50
4	41	Private	264663	Some-college	10	Separated	Prof-specialty	Own-child	White	Female	0	3900	40	United-States	<=50

Figure 2: Displaying first 5 enteries

5. Dataset shape

```
df.shape
```

```
(32561, 15)
```

5. Handle missing values

```
print((df == "?").sum())

df = df.replace("?", np.nan)

df = df.dropna()
```

```
age      0
workclass 0
fnlwgt   0
education 0
education.num 0
marital.status 0
occupation 0
relationship 0
race      0
sex       0
capital.gain 0
capital.loss 0
hours.per.week 0
native.country 0
income    0
dtype: int64
```

Figure 3: Displaying removed missing values

7. Encode the Target Column

```
df_encoded = df.copy()

le_target = LabelEncoder()

df_encoded["income"] = le_target.fit_transform(
    df_encoded["income"]
)

print("\nEncoded Target Classes:")

print(le_target.classes_)

Encoded Target Classes:
['<=50K' '>50K']
```

Figure 4: Distribution encoded target classes

8. Encode Categorical Feature Columns

```
categorical_cols = df_encoded.select_dtypes(
    include="object"
).columns.tolist()

print("\nCategorical Columns Being Encoded:")
```

```
print(categorical_cols)

df_encoded = pd.get_dummies(
    df_encoded,
    columns=categorical_cols,
    drop_first=True
)

Categorical Columns Being Encoded:
['workclass', 'education', 'marital.status', 'occupation', 'relationship', 'race', 'sex', 'native.country']
```

Figure 5: Displaying encoded categorical columns

9. Separate Independent (X) and Dependent (y) Variables

```
X = df_encoded.drop(columns=["income"])
y = df_encoded["income"]

print("\nIndependent Variables (X) Shape:", X.shape)
print("Dependent Variable (y) Shape:", y.shape)

Independent Variables (X) Shape: (30162, 96)
Dependent Variable (y) Shape: (30162,)
```

Figure 6: Separation of X and Y

10. Split Dataset into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.60,
    random_state=45,
    stratify=y
)

print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape :", X_test.shape)
```

Training Data Shape: (12064, 96)
Testing Data Shape : (18098, 96)

Figure 7: Testing and training data distribution

11. Feature Scaling

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

12. Logistic Regression Model

```
logistic_model = LogisticRegression(
```

```
    max_iter=1000
```

```
)
```

```
logistic_model.fit(
```

```
    X_train_scaled,
```

```
    y_train
```

```
)
```

```
logistic_pred = logistic_model.predict(
```

```
    X_test_scaled
```

```
)
```

```
logistic_accuracy = accuracy_score(
```

```
    y_test,
```

```
    logistic_pred
```

```
)
```

```
logistic_precision = precision_score(
```

```
    y_test,
```

```
    logistic_pred
```

```
)
```

```
logistic_recall = recall_score(
```

```
y_test,  
logistic_pred  
)  
logistic_f1 = f1_score(  
    y_test,  
    logistic_pred  
)  
print("Accuracy :", logistic_accuracy)  
print("Precision :", logistic_precision)  
print("Recall   :", logistic_recall)  
print("F1 Score :", logistic_f1)
```

```
Accuracy   : 0.847994253508675  
Precision  : 0.7358795051102743  
Recall     : 0.6073251942286348  
F1 Score   : 0.6654505654870485
```

Figure 8: Result of logistic regression model

```
logistic_cm = confusion_matrix(  
    y_test,  
    logistic_pred  
)  
disp_log = ConfusionMatrixDisplay(  
    confusion_matrix=logistic_cm,  
    display_labels=le_target.classes_  
)  
disp_log.plot(cmap="Blues")  
plt.title("Confusion Matrix - Logistic Regression")  
plt.show()
```

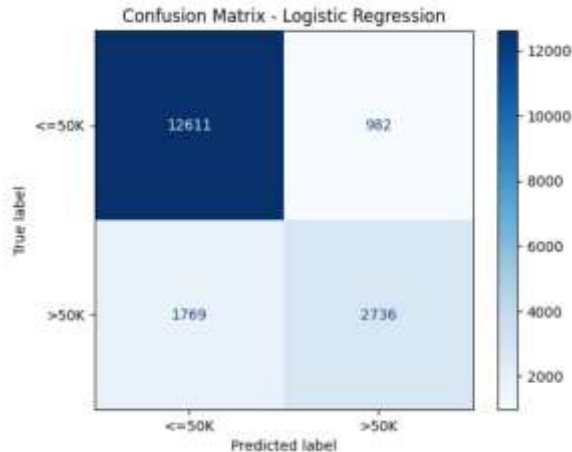


Figure 9: Confusion matrix for logistic regression

13. KNN model

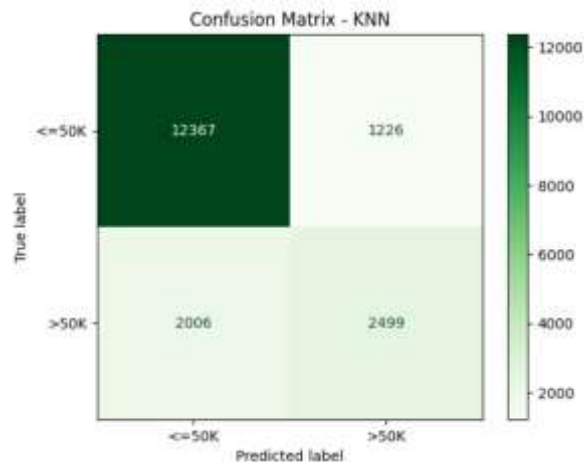
```

knn_model = KNeighborsClassifier(
    n_neighbors=15
)
knn_model.fit(
    X_train_scaled,
    y_train
)
knn_pred = knn_model.predict(
    X_test_scaled
)
knn_accuracy = accuracy_score(
    y_test,
    knn_pred
)
knn_precision = precision_score(
    y_test,
    knn_pred
)
knn_recall = recall_score(
    y_test,
    knn_pred
)
knn_f1 = f1_score(
    y_test,
    knn_pred
)
print("Accuracy :", knn_accuracy)
print("Precision :", knn_precision)
print("Recall   :", knn_recall)
print("F1 Score :", knn_f1)

```

```
Accuracy : 0.8214167311305116
Precision : 0.6708724832214765
Recall : 0.5547169811320755
F1 Score : 0.6072904009720534
```

```
knn_cm = confusion_matrix(
    y_test,
    knn_pred
)
disp_knn = ConfusionMatrixDisplay(
    confusion_matrix=knn_cm,
    display_labels=le_target.classes_
)
disp_knn.plot(cmap="Greens")
plt.title("Confusion Matrix - KNN")
plt.show()
```



+

14. Decision tree model

```
dt_model = DecisionTreeClassifier(
    max_depth=8,
    random_state=45
)
dt_model.fit(
    X_train,
    y_train
)
dt_pred = dt_model.predict(
    X_test
)
dt_accuracy = accuracy_score(
    y_test,
    dt_pred
)
```

```
dt_precision = precision_score(
    y_test,
    dt_pred
)
dt_recall = recall_score(
    y_test,
    dt_pred
)
dt_f1 = f1_score(
    y_test,
    dt_pred
)
print("Accuracy :", dt_accuracy)
print("Precision :", dt_precision)
print("Recall   :", dt_recall)
print("F1 Score :", dt_f1)

Accuracy   : 0.8494308763399271
Precision  : 0.7902804957599479
Recall     : 0.5378468368479468
F1 Score   : 0.6400739664509312
```

Figure 10: Result of KNN Model

```
dt_cm = confusion_matrix(
    y_test,
    dt_pred)
disp_dt = ConfusionMatrixDisplay(
    confusion_matrix=dt_cm,
    display_labels=le_target.classes_)
disp_dt.plot(cmap="Oranges")
plt.title("Confusion Matrix - Decision Tree")
plt.show()
```

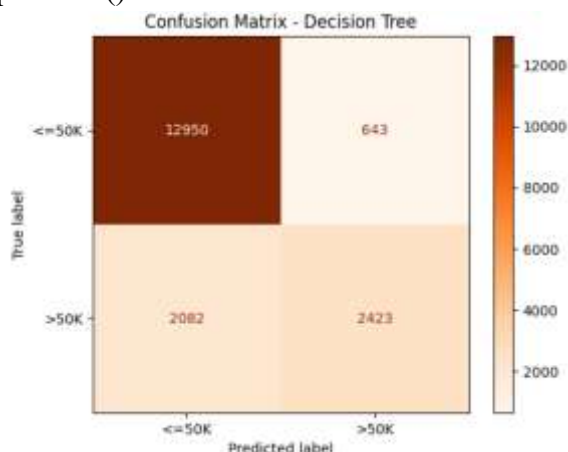


Figure 11: Confusion Matrix for KNN Model

15. Compare All Three Models

```
comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "KNN",
        "Decision Tree" ],
    "Accuracy": [
        round(logistic_accuracy * 100, 2),
        round(knn_accuracy * 100, 2),
        round(dt_accuracy * 100, 2) ],
    "Precision": [
        round(logistic_precision * 100, 2),
        round(knn_precision * 100, 2),
        round(dt_precision * 100, 2) ],
    "Recall": [
        round(logistic_recall * 100, 2),
        round(knn_recall * 100, 2),
        round(dt_recall * 100, 2) ],
    "F1 Score": [
        round(logistic_f1 * 100, 2),
        round(knn_f1 * 100, 2),
        round(dt_f1 * 100, 2) ]
})
print(comparison.to_string(index=False))
```

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	84.80	73.59	60.73	66.55
KNN	82.14	67.09	55.47	60.73
Decision Tree	84.95	79.09	53.74	64.00

Figure 12: Comparison Table for models

15. Comparison Graph

```
metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
logistic_scores = [
    logistic_accuracy * 100,
    logistic_precision * 100,
    logistic_recall * 100,
    logistic_f1 * 100
]
knn_scores = [
    knn_accuracy * 100,
    knn_precision * 100,
    knn_recall * 100,
    knn_f1 * 100
]
dt_scores = [
```

```

dt_accuracy * 100,
dt_precision * 100,
dt_recall * 100,
dt_f1 * 100
]
x = np.arange(len(metrics))
width = 0.25
plt.figure(figsize=(10, 6))
plt.bar(x - width, logistic_scores, width, label="Logistic Regression")
plt.bar(x, knn_scores, width, label="KNN")
plt.bar(x + width, dt_scores, width, label="Decision Tree")
plt.xlabel("Metrics")
plt.ylabel("Score (%)")
plt.title("Logistic Regression vs KNN vs Decision Tree Comparison")
plt.xticks(x, metrics)
plt.ylim(0, 110)
plt.legend()
plt.tight_layout()
plt.show()

```

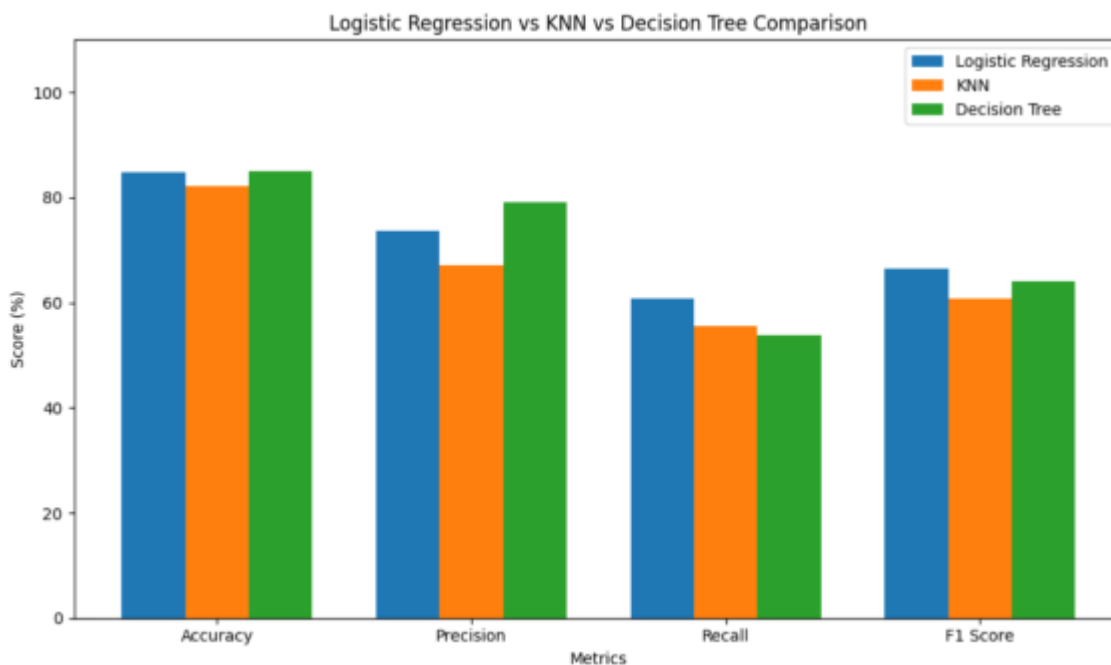


Figure 13: Comparison graph for Logistic regression, KNN Model and Decision tree

Learning Outcomes (What I Have Learnt)

1. Understand Logistic Regression, k-NN, and Decision Tree.
2. Preprocess and train classification models.
3. Evaluate using Confusion Matrix, Precision, Recall, F1-Score.
4. Compare models and pick the best.



UNIVERSITY INSTITUTE *of*
COMPUTING
Asia's Fastest Growing University

